Task: Build a Full-Stack ML Prediction App (Proof-of-Concept)

To: Shams **From:** Mostafa Badr **Date:** 2025-11-17

---

1. Objective

The goal of this task is to build a *simple*, end-to-end application that connects a web frontend (HTML/JS/CSS) to a Python backend for a live machine learning prediction.

This is a foundational task to prove we have the "plumbing" right for the main project. We are testing the *connection* from the user's browser, to the Python server, into the ML model, and back to the user.

2. The Stack

- **Frontend:** A single `index.html` file containing pure HTML, CSS (in a `<style>` tag), and JavaScript (in a `<script>` tag).
- **Backend:** A simple Python server using the **Flask** framework.
- **Model:** A `scikit-learn` model trained in a separate script and saved to a file using `joblib`.

---

3. Phase 1: The Model (`train.py`)

First, you must train a simple model and save it so your server can use it. Create a new file called `train.py`.

1. **Data:** Download the "Car Price Prediction Dataset" from Kaggle. We will use the `processes2.csv` file.
    - *Kaggle Source:* https://www.kaggle.com/datasets/jacksondivakarr/sample34
2. **Features (Keep it simple):**
    - Load the CSV into a `pandas` DataFrame.
    - Keep only these columns: `year`, `km_driven`, `fuel`, `transmission`, and the target `selling_price`.
3. **Preprocessing:**
    - The features `fuel` and `transmission` are categorical. You must convert them to numbers.
    - Use `scikit-learn`'s `OneHotEncoder` for this.
    - **Important:** You must save this encoder. The server will need it to transform the live user input.
4. **Model:**
    - Train a simple `LinearRegression` model using the preprocessed data.
5. **Save Artifacts:**
    - Use the `joblib` library to save your trained model and your fitted encoder to files.
    - `joblib.dump(model, 'model.joblib')`
    - `joblib.dump(encoder, 'encoder.joblib')`

At the end of this phase, you should have `model.joblib` and `encoder.joblib`.

4. Phase 2: The Backend API (`app.py`)

Next, create the Python web server. Create a file called `app.py`.

1. **Setup:**
    - Install Flask: `pip install Flask flask-cors`
    - Import `Flask`, `jsonify`, `request` from `flask`, `joblib`, `pandas`, and `numpy`.
    - Enable CORS: `CORS(app)` (This is critical to allow your HTML file to talk to your server).
2. **Load Artifacts:**
    - Load your saved models *outside* the request functions (so it only happens once on startup):
        - `model = joblib.load('model.joblib')`
        - `encoder = joblib.load('encoder.joblib')`
3. **Create `/predict` Endpoint:**
    - Create a single API route that listens at `/predict` and only accepts `POST` requests.
    - Inside this function, do the following:
        1. Get the incoming JSON data from the user: `data = request.json`
        2. Convert this JSON into a `pandas` DataFrame with the correct columns: `year`, `km_driven`, `fuel`, `transmission`.
        3. Use your loaded `encoder` to transform the categorical features.
        4. Use your loaded `model` to make a prediction on the transformed data.
        5. Get the single prediction value (e.g., `prediction[0]`).
        6. Return the prediction as a JSON response: `return jsonify({'predicted_price': prediction_value})`

5. Phase 3: The Frontend (`index.html`)

Create a single file named `index.html`. This file will contain all your HTML, CSS, and JS.

1. **HTML (The Form):**
    - Create a form with a `<h2>` title like "Used Car Price Predictor".
    - Add inputs for the features your model needs:
        - `Year:` (number input)
        - `Kilometers Driven:` (number input)
        - `Fuel Type:` (a `<select>` dropdown with "Petrol", "Diesel", "CNG")
        - `Transmission:` (a `<select>` dropdown with "Manual", "Automatic")
    - Add a `<button>` with the ID `predict-button`.
    - Add an `<h2>` tag to show the result, e.g., `<h2 id="result">Predicted Price: ...</h2>`.
2. **CSS (The Style):**
    - Add a `<style>` tag in the `<head>`.
    - Add some *basic* styling to make it look clean (e.g., center the content, style the button).
3. **JavaScript (The Logic):**
    - Add a `<script>` tag at the *bottom* of the `<body>`.
    - Add an event listener to your button (`document.getElementById('predict-button')...`).
    - Inside the click event: a. Prevent the form's default submit action (`event.preventDefault()`). b. Get the values from all your form inputs. c. Create a JavaScript object with these values (e.g., `const inputData = { year: 2018, km_driven: 50000, ... }`). d. Use the `fetch()` API to send this object to your Python server (e.g., `fetch('http://127.0.0.1:5000/predict', ...)`). e. **Method:** Use `POST`. f. **Headers:** Set `'Content-Type': 'application/json'`. g. **Body:** Send your `inputData` object, but run it through `JSON.stringify()`. h. Use `.then(response`

`=> response.json())` to get the result. i. Use `.then(data => { ... })` to update the `innerHTML` of your `result` `<h2>` with the price from `data.predicted_price`.

6. Success Criteria

The task is complete when:

1. You can run `python train.py` once to create the model files.
2. You can run `python app.py` to start the server.
3. You can open the `index.html` file in your browser.
4. You can fill in the form, click "Predict", and see the predicted price appear on the page without the page reloading.

This task is a perfect "first step" as it directly simulates the architecture of the final project, confirming the student's ability to handle the full-stack data flow.

This video demonstrates how to save and load scikit-learn models using `joblib`, which is a key part of this task. Save and Load ML Models with Joblib

---